

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스 소개

DAY1	DAY2	DAY3	DAY4	DAY5
데이터 분석을 위한 기초 수학			데이터 집계 및 시각화	
DAY6	DAY7	DAY8	DAY9	DAY10
	미니 프로젝트	확률분포, AB테스트		
DAY11	DAY12	DAY13	DAY14	DAY15
머신러닝				
DAY16	DAY17	DAY18	DAY19	DAY20
딥러닝		파이널 프로젝트		

□ 확률변수

□ 조건부확률

□ 평균, 분산, 표준편차

□ 모집단과 표본

□ 공분산/상관계수

#

AI/BigData 인텐시브 과정

9
Section 1. 기초 통계

Section 1-1. 확률 변수

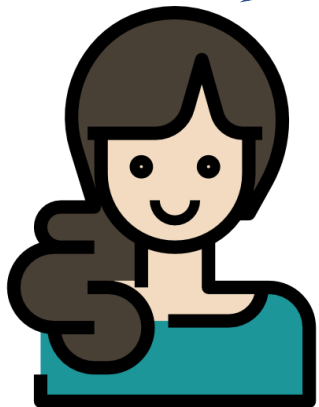
확률 변수(random variable)

확률 변수(random variable): 확률 현상을 통해 값이 확률적으로 정해지는 변수

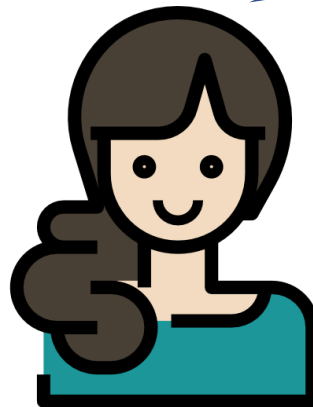
확률 현상

발생 가능한 결과들은 알지만 가능한 결과들 중 어떤 결과가 나올지는 모르는 현상

동전 던지면 앞면 아니면 뒷면이
나오는건 확실해



그런데 앞면/뒷면 중 뭐가 나올지는 몰라!



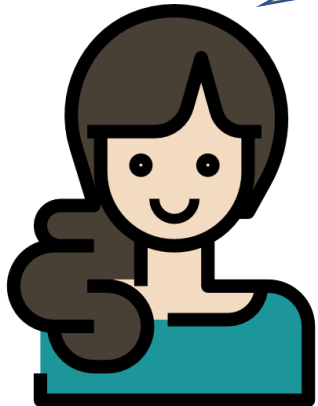
확률

이론적 확률 vs 경험적 확률

이론적 확률 vs 경험적 확률

이론적 확률 – 이론적(theoretical), 미래(future) *미래에 발생할 사건에 대한 확률*

동전을 던지면 앞면이 나올 확률은 ?



$$\text{확률} = \frac{\text{관심 대상}}{\text{나올수 있는 결과 갯수}}$$

$$\text{앞면이 나올 확률} = \frac{\text{앞면}}{\text{앞면} + \text{뒷면}} = \frac{1}{2} = 50\%$$

이론적 확률 vs 경험적 확률

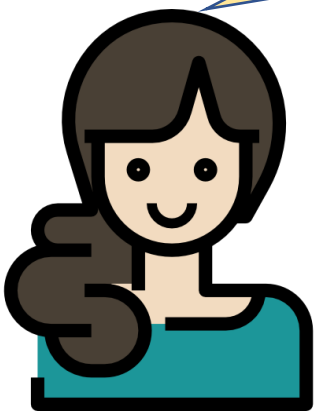
경험적 확률 – 경험적(empirical), 과거(past)

이미 발생한 과거 사건에 대한 확률

많이 동전 던지면 이론적 확률에 가까워짐

과거형

동전을 100번 던졌더니
앞면이 60번, 뒷면이 40번 나왔어



$$\text{경험적 확률} = \frac{\text{관심 대상}}{\text{전체 데이터 수}}$$

$$\text{앞면의 비율} = \frac{60}{100} = \frac{3}{5} = 60\%$$

확률 변수 vs 상수 변하지 X 구분 필요!

상수

10

상수

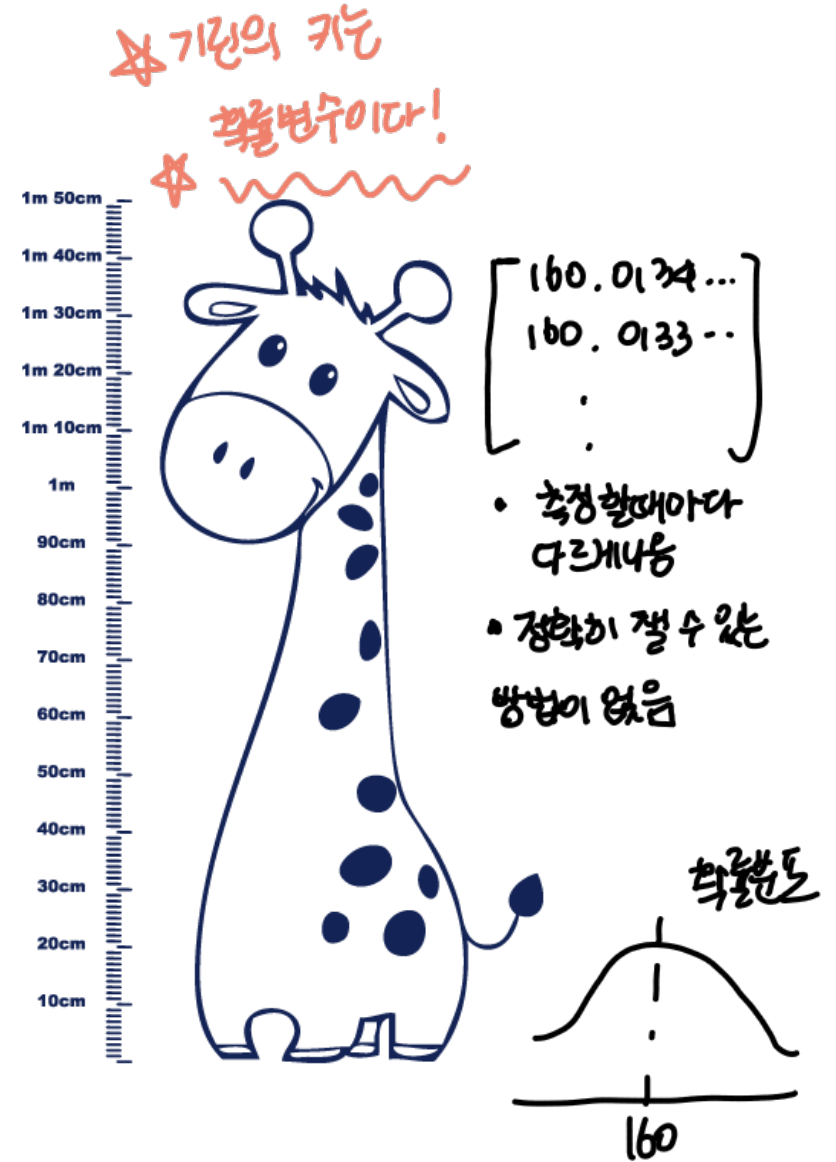
7

상수

π
3.14
15926

확률 변수

x



확률의 정의

확률: 어떤 사건이 발생할 가능성을 수치화 시킨 것으로 0과 1사이의 값을 가짐

- 동전을 던졌을 때 앞면이 나올 확률은?
- 주사위를 던졌을 때 6이 나올 확률은?
- 게임 유저가 다음 주에 접속할 확률은?

확률의 공리적 정의

1) $0 \leq P(A) \leq 1$ $\rightarrow$ indication func. 이므로 포기할수있음.

2) $P(\Omega) = 1$ $\Omega =$ 표본공간

3) 만약 $A_1, A_2, \dots$ 가 상호 배반 사건이면, $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$

표본 공간(sample space) (Ω)

- 동전 던졌을 때, $\Omega = \{\text{앞면, 뒷면}\}$
- 주사위를 던졌을 때, $\Omega = \{1, 2, 3, 4, 5, 6\}$
- 상품권 추첨할 때, $\Omega = \{\text{당첨, 미당첨}\}$

AI/BigData 인텐시브 과정

Section 1. 기초 통계

Section 1-2. 조건부 확률

확률의 독립(independent)

좌변과 우변의 값이 ^{같음} 같으면 독립

같지 않으면 독립이 아님

$$P(A \cap B) = P(A)P(B)$$

사건 A와 사건 B가 동시에 발생할 확률

사건 A가 발생할 확률 X 사건 B가 발생할 확률

확률의 배반(disjoint)

$$P(A \cap B) = 0$$

즉, 배반 사건이므로, $P(A \cap B) = 0$

사건 A와 사건 B가 동시에 발생할 확률이 0

조건부 확률

확률 변수: 대문자 평문.

베이시안

사건 Y와 사건 X가 동시에 발생할 확률

Y given X
라고 읽음

$$P(Y|X) = \frac{P(Y \cap X)}{P(X)}$$

사건 X가 발생할 확률

사건 X가 발생했을 때, 사건 Y가 발생할 확률
조건

조건부 확률

$$P(Y|X) = \frac{P(Y \cap X)}{P(X)}$$

$$P(Y \cap X) = P(Y|X)P(X) = P(X|Y)P(Y)$$

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

조건부 확률

$$P(X|Y) = \frac{P(X \cap Y)}{P(Y)}$$

$$P(X \cap Y) = P(X|Y)P(Y)$$

사전확률(Prior Probability) : 이벤트 발생(前) 확률

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

사후확률(Posterior Probability) : 이벤트 발생(后) 확률
→ 이것을 구하는게 목적

조건부 확률

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

core function

scaler

X는 Y에 영향을 받지 않으므로 상수 취급

$$\propto P(X|Y)P(Y)$$

분자만 보기도..

외계어

조건부 확률(X와 Y가 독립인 경우)

$$P(Y|X) = \frac{P(Y \cap X)}{P(X)} = \frac{P(Y)P(X)}{P(X)} = P(Y)$$

$$P(Y|X) = P(Y)$$

조건 X가 발생하든 안하든 확률이 똑같다.
조건 X가 영향을 미치지 않는다

방의 개수(RM)가 7개 초과일 확률

$$P(\text{방의 개수} > 7) = ?$$

방의 개수(RM)가 7개 초과일 확률

```
In [1]: 1 import numpy as np
        2 import pandas as pd
```

```
In [2]: 1 df = pd.read_csv("../data/house_prices.csv", encoding='cp949')
```

```
In [3]: 1 df
```

Out[3]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

방의 개수(RM)가 7개 초과일 확률

```
In [10]: 1 rm = df[df['RM'] > 7]
          2 rm
```

Out[10]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
2	0.02729	0.0	7.07	0	0.4690	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
4	0.06905	0.0	2.18	0	0.4580	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
40	0.03359	75.0	2.95	0	0.4280	7.024	15.8	5.4011	3	252	18.3	395.62	1.98	34.9
55	0.01311	90.0	1.22	0	0.4030	7.249	21.9	8.6966	5	226	17.9	395.93	4.81	35.4
64	0.01951	17.5	1.38	0	0.4161	7.104	59.5	9.2229	3	216	18.6	393.24	8.05	33.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
364	3.47428	0.0	18.10	1	0.7180	8.780	82.9	1.9047	24	666	20.2	354.55	5.29	21.9
370	6.53876	0.0	18.10	1	0.6310	7.016	97.5	1.2024	24	666	20.2	392.05	2.96	50.0
375	19.60910	0.0	18.10	0	0.6710	7.313	97.9	1.3163	24	666	20.2	396.90	13.44	15.0
453	8.24809	0.0	18.10	0	0.7130	7.393	99.3	2.4527	24	666	20.2	375.87	16.74	17.8
482	5.73116	0.0	18.10	0	0.5320	7.061	77.0	3.4106	24	666	20.2	395.28	7.01	25.0

64 rows × 14 columns

```
In [11]: 1 n_rm = len(rm)
          2 print(n_rm)
```

64

```
In [13]: 1 n = len(df)
          2 print(n)
```

506

```
In [14]: 1 p = n_rm/n
          2 print(p)
```

0.12648221343873517

조건부 확률

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

$$P(\text{방 개수} > 7 \mid \text{학생 교사 비율} < 15) = \frac{P((\text{방 개수} > 7) \& (\text{학생 교사 비율} < 15))}{P(\text{학생 교사 비율} < 15)}$$

조건부 확률

$P(\text{학생 교사 비율} < 15) =$

```
In [22]: 1 ptratio = df[df['PTRATIO'] < 15]
         2 ptratio.head(10)
```

Out[22]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
142	3.32105	0.0	19.58	1	0.871	5.403	100.0	1.3216	5	403	14.7	396.90	26.82	13.4
143	4.09740	0.0	19.58	0	0.871	5.468	100.0	1.4118	5	403	14.7	396.90	26.42	15.6
144	2.77974	0.0	19.58	0	0.871	4.903	97.8	1.3459	5	403	14.7	396.90	29.29	11.8
145	2.37934	0.0	19.58	0	0.871	6.130	100.0	1.4191	5	403	14.7	172.91	27.80	13.8
146	2.15505	0.0	19.58	0	0.871	5.628	100.0	1.5166	5	403	14.7	169.27	16.65	15.6
147	2.36862	0.0	19.58	0	0.871	4.926	95.7	1.4608	5	403	14.7	391.71	29.53	14.6
148	2.33099	0.0	19.58	0	0.871	5.186	93.8	1.5296	5	403	14.7	356.99	28.32	17.8
149	2.73397	0.0	19.58	0	0.871	5.597	94.9	1.5257	5	403	14.7	351.85	21.45	15.4
150	1.65660	0.0	19.58	0	0.871	6.122	97.3	1.6180	5	403	14.7	372.80	14.10	21.5
151	1.49632	0.0	19.58	0	0.871	5.404	100.0	1.5916	5	403	14.7	341.60	13.28	19.6

```
In [30]: 1 p_ptratio = len(ptratio)/len(df)
         2 print(p_ptratio)
```

0.11462450592885376

조건부 확률

$$P((\text{방 개수} > 7) \& (\text{학생 교사 비율} < 15))$$

```
In [28]: 1 ptratio_rm = df[ (df['RM'] > 7) & (df['PTRATIO'] < 15) ]  
        2 ptratio_rm.head(10)
```

Out[28]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
161	1.46336	0.0	19.58	0	0.6050	7.489	90.8	1.9709	5	403	14.7	374.43	1.73	50.0
162	1.83377	0.0	19.58	1	0.6050	7.802	98.2	2.0407	5	403	14.7	389.61	1.92	50.0
163	1.51902	0.0	19.58	1	0.6050	8.375	93.9	2.1620	5	403	14.7	388.45	3.32	50.0
166	2.01019	0.0	19.58	0	0.6050	7.929	96.2	2.0459	5	403	14.7	369.30	3.70	50.0
195	0.01381	80.0	0.46	0	0.4220	7.875	32.0	5.6484	4	255	14.4	394.23	2.97	50.0
196	0.04011	80.0	1.52	0	0.4040	7.287	34.1	7.3090	2	329	12.6	396.90	4.08	33.3
197	0.04666	80.0	1.52	0	0.4040	7.107	36.6	7.3090	2	329	12.6	354.31	8.61	30.3
198	0.03768	80.0	1.52	0	0.4040	7.274	38.3	7.3090	2	329	12.6	392.20	6.62	34.6
202	0.02177	82.5	2.03	0	0.4150	7.610	15.7	6.2700	2	348	14.7	395.38	3.11	42.3
203	0.03510	95.0	2.68	0	0.4161	7.853	33.2	5.1180	4	224	14.7	392.78	3.81	48.5

```
In [31]: 1 p_ptratio_rm = len(ptratio_rm)/len(df)  
        2 print(p_ptratio_rm)
```

0.04940711462450593

조건부 확률

→ 경험적 확률임. 데이터기반으로 구함으로.

이론적 확률
경험적 확률

$$P(\text{방 개수} > 7 \mid \text{학생 교사 비율} < 15) = \frac{P((\text{방 개수} > 7) \& (\text{학생 교사 비율} < 15))}{P(\text{학생 교사 비율} < 15)}$$

In [33]:



```
1 p = p_ptratio_rm / p_ptratio
2 print(p)
```

0.43103448275862066

AI/BigData 인텐시브 과정

Section 1. 기초 통계

Section 1-3. 평균, 분산, 표준 편차

평균

수열의 평균

(이미 데이터가 있는 경우)

$$\bar{X} = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

(아직 데이터가 없는 경우)

$$= \frac{1}{n} \sum_{i=1}^n x_i$$

$$\bar{X} = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

대수적
평균인 경우
이항평균

평균

관하는 이유) 데이터의 '위치'를 알려주기 때문. → 평균은 위치모수이다

1, 3, 5, 9, 12

$$\bar{x} = \frac{1 + 3 + 5 + 9 + 12}{5} = \frac{30}{5} = 6$$

위치 모수(location parameter)

분산 & 표준 편차

데이터가 평균으로부터 얼마나 떨어져있는지를 보는 것.

제곱이 없으면 계산결과 0.

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

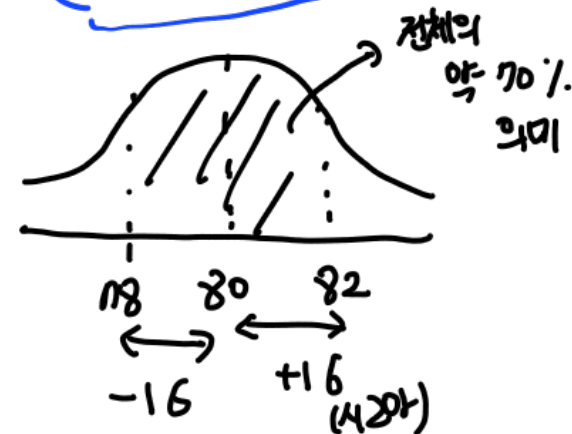
평균 80점
분산 4점²

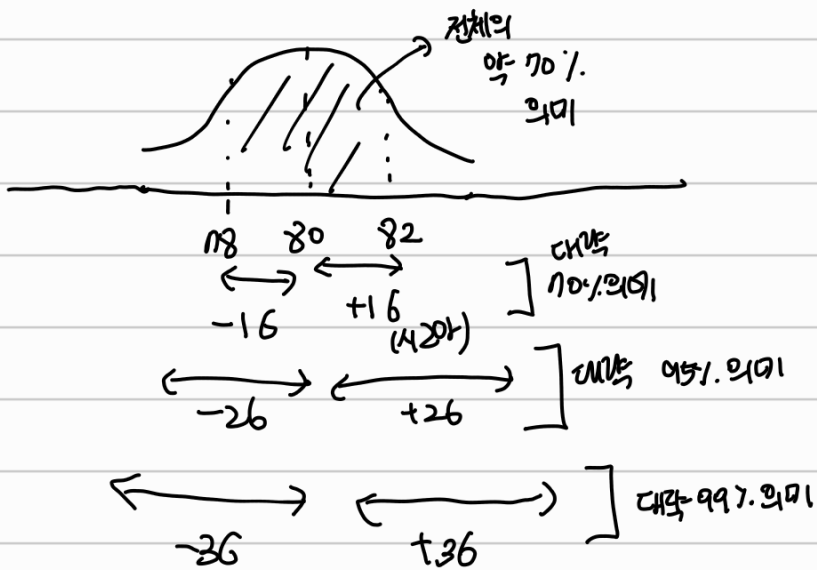
표준편차 2점

$$s = \sqrt{s^2}$$

얼마나 퍼져있는지 scale을 알려준다

scale parameter





① $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$

제약이 있

자유도

제약조건이 늘어날수록 더 바뀔

제약조건이 개수만큼 바뀔

Q. 분산을 알기 위해서는 평균값 하나만 알아야 하니까?

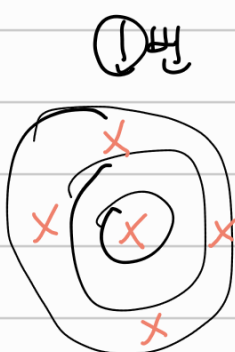
$$s = \sqrt{s^2}$$

얼마 퍼져있는지 scale을 알려준다

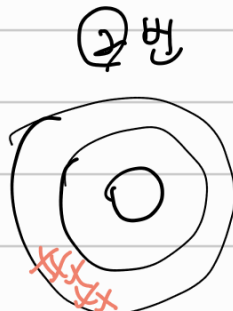
scale parameter

사실에서는 자유도 알려주면 그대로.

② $s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \Rightarrow$ 이것도 맞아서 이거 쓰도 될.



unbiased : good
efficiency : bad



unbiased : bad (10점을 못맞췄어)
efficiency : good (올려놔서?)

이런 느낌? 장단점이 있다는 뜻인듯

분산 & 표준 편차

1, 3, 5, 9, 12

$$\bar{x} = 6$$

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

$$= \frac{1}{5-1} \{(1-6)^2 + (3-6)^2 + (5-6)^2 + (9-6)^2 + (12-6)^2\}$$

$$= \frac{1}{4} \{25 + 9 + 1 + 9 + 36\} = \frac{1}{4} \times 80$$

$$= 20$$

$$s^2 = 20$$

$$s = \sqrt{20}$$

자유도(degree of freedom)의 개념

$n-1$

1, 3, 4, 6, x

무엇을 알고, 자유가 있나.

자유도(degree of freedom)의 개념

1, 3, 4, 6, x



6

$$\bar{x} = 4$$

이렇게 평균이 4가 되어야 할까?

기는 6이 필수밖에 없음.

→ 6은 자유가 없음

자유도(degree of freedom)의 개념

자유가 있는지 없는지

$$x_1, x_2, x_3, x_4, x_5$$

→ 자유로운 숫자는 5개.

→ 자유도가 5다.

자유도(degree of freedom)의 개념

 x_1, x_2, x_3, x_4, x_5

여기까지 자유도
아무거나 들어갈 수 있음

기. ~ 기.가
정해지면
이제는 값을
입을 수 없음

자유도 = 4. (1-1?)

 $\bar{x} = 3$

Q. 자유도가 몇 일까?

평균

```
In [37]: ▶ 1 np.mean(df['RM'])
```

```
Out[37]: 6.284634387351779
```

```
In [38]: ▶ 1 df['RM'].mean()
```

```
Out[38]: 6.284634387351779
```

```
In [54]: ▶ 1 n = len(df)
2 sum_x = 0
3 for x in df['RM']:
4     sum_x += x
5 mean_rm = sum_x/n
6 print(mean_rm)
```

```
6.284634387351787
```

$$\bar{X} = \frac{x_1 + x_2 + \cdots + x_n}{n} = \frac{1}{n} \sum_{i=1}^n x_i$$

분산, 표준 편차

numpy의 경우

```
In [39]: 1 np.var(df['RM'])
```

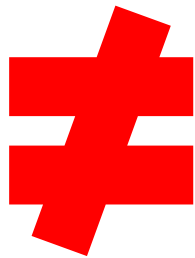
```
Out[39]: 0.49269521612976297
```

```
In [41]: 1 np.std(df['RM'])
```

```
Out[41]: 0.7019225143345689
```

n으로 나눈다

$$s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$



pandas의 경우

```
In [40]: 1 df['RM'].var()
```

```
Out[40]: 0.49367085022110907
```

```
In [42]: 1 df['RM'].std()
```

```
Out[42]: 0.7026171434153233
```

n-1로 나눈다.

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

```
1 n = len(df)
2 ss = 0
3 for x in df['RM']:
4     ss += (x - mean_rm)**2
```

```
1 var_rm = ss/n
2 std_rm = var_rm**0.5
3 print(var_rm)
4 print(std_rm)
```

```
0.49269521612976347
```

```
0.7019225143345692
```

```
1 var_rm = ss/(n-1)
2 std_rm = var_rm**0.5
3 print(var_rm)
4 print(std_rm)
```

```
0.4936708502211095
```

```
0.7026171434153237
```

numpy, pandas 차이

```
In [65]: 1 df['RM'].var()
```

```
Out[65]: 0.49367085022110907
```

```
In [66]: 1 df['RM'].var(ddof=1)
```

```
Out[66]: 0.49367085022110907
```

```
In [67]: 1 df['RM'].var(ddof=0)
```

```
Out[67]: 0.49269521612976297
```

내가
자유도를
정해줘야.
00(강)만 넣자.
2.3(x)

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

pandas.DataFrame.var

`DataFrame.var(axis=None, skipna=True, level=None, ddof=1, numeric_only=None, **kwargs)` [\[source\]](#)

Return unbiased variance over requested axis.

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters: `axis` : {index (0), columns (1)}

For Series this parameter is unused and defaults to 0.

skipna : bool, default True

Exclude NA/null values. If an entire row/column is NA, the result will be NA.

level : int or level name, default None

If the axis is a MultiIndex (hierarchical), count along a particular level, collapsing into a Series.

! *Deprecated since version 1.3.0:* The level keyword is deprecated. Use groupby instead.

ddof : int, default 1

Delta Degrees of Freedom. The divisor used in calculations is $N - \text{ddof}$, where N represents the number of elements.

계산 N-1
가분자!

numeric_only : bool, default None

Include only float, int, boolean columns. If None, will attempt to use everything, then use only numeric data. Not implemented for Series.

numpy, pandas 차이

```
In [68]: 1 df['RM'].std()
```

```
Out[68]: 0.7026171434153233
```

```
In [69]: 1 df['RM'].std(ddof=1)
```

```
Out[69]: 0.7026171434153233
```

```
In [70]: 1 df['RM'].std(ddof=0)
```

```
Out[70]: 0.7019225143345689
```

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

pandas.DataFrame.std

`DataFrame.std(axis=None, skipna=True, level=None, ddof=1, numeric_only=None, **kwargs)` [\[source\]](#)

Return sample standard deviation over requested axis.

Normalized by N-1 by default. This can be changed using the ddof argument.

Parameters: **axis** : {index (0), columns (1)}

For *Series* this parameter is unused and defaults to 0.

skipna : bool, default True

Exclude NA/null values. If an entire row/column is NA, the result will be NA.

level : int or level name, default None

If the axis is a MultiIndex (hierarchical), count along a particular level, collapsing into a *Series*.

! *Deprecated since version 1.3.0:* The level keyword is deprecated. Use *groupby* instead.

ddof : int, default 1

Delta Degrees of Freedom. The divisor used in calculations is N - ddof, where N represents the number of elements.

numeric_only : bool, default None

Include only float, int, boolean columns. If None, will attempt to use everything, then use only numeric data. Not implemented for *Series*.

numpy, pandas 차이

```
In [71]: 1 np.var(df['RM'])
```

```
Out[71]: 0.49269521612976297
```

```
In [72]: 1 np.var(df['RM'], ddof=0)
```

```
Out[72]: 0.49269521612976297
```

```
In [73]: 1 np.var(df['RM'], ddof=1)
```

```
Out[73]: 0.49367085022110907
```

$$s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

numpy.var

`numpy.var(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>, *, where=<no value>)` [\[source\]](#)

Compute the variance along the specified axis.

Returns the variance of the array elements, a measure of the spread of a distribution. The variance is computed for the flattened array by default, otherwise over the specified axis.

Parameters: *a* : *array_like*

Array containing numbers whose variance is desired. If *a* is not an array, a conversion is attempted.

axis : *None or int or tuple of ints, optional*

Axis or axes along which the variance is computed. The default is to compute the variance of the flattened array.

! New in version 1.7.0.

If this is a tuple of ints, a variance is performed over multiple axes, instead of a single axis or all the axes as before.

dtype : *data-type, optional*

Type to use in computing the variance. For arrays of integer type the default is `float64`; for arrays of float types it is the same as the array type.

out : *ndarray, optional*

Alternate output array in which to place the result. It must have the same shape as the expected output, but the type is cast if necessary.

ddof : *int, optional*

“Delta Degrees of Freedom”: the divisor used in the calculation is $N - \text{ddof}$, where N represents the number of elements. By default *ddof* is zero.

numpy, pandas 차이

```
In [74]: ▶ 1 np.std(df['RM'])
```

```
Out[74]: 0.7019225143345689
```

```
In [75]: ▶ 1 np.std(df['RM'], ddof=0)
```

```
Out[75]: 0.7019225143345689
```

```
In [76]: ▶ 1 np.std(df['RM'], ddof=1)
```

```
Out[76]: 0.7026171434153233
```

$$s^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

numpy.std

`numpy.std(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>, *, where=<no value>)` [\[source\]](#)

Compute the standard deviation along the specified axis.

Returns the standard deviation, a measure of the spread of a distribution, of the array elements. The standard deviation is computed for the flattened array by default, otherwise over the specified axis.

Parameters: `a` : *array_like*

Calculate the standard deviation of these values.

`axis` : *None or int or tuple of ints, optional*

Axis or axes along which the standard deviation is computed. The default is to compute the standard deviation of the flattened array.

 *New in version 1.7.0.*

If this is a tuple of ints, a standard deviation is performed over multiple axes, instead of a single axis or all the axes as before.

`dtype` : *dtype, optional*

Type to use in computing the standard deviation. For arrays of integer type the default is float64, for arrays of float types it is the same as the array type.

`out` : *ndarray, optional*

Alternative output array in which to place the result. It must have the same shape as the expected output but the type (of the calculated values) will be cast if necessary.

`ddof` : *int, optional*

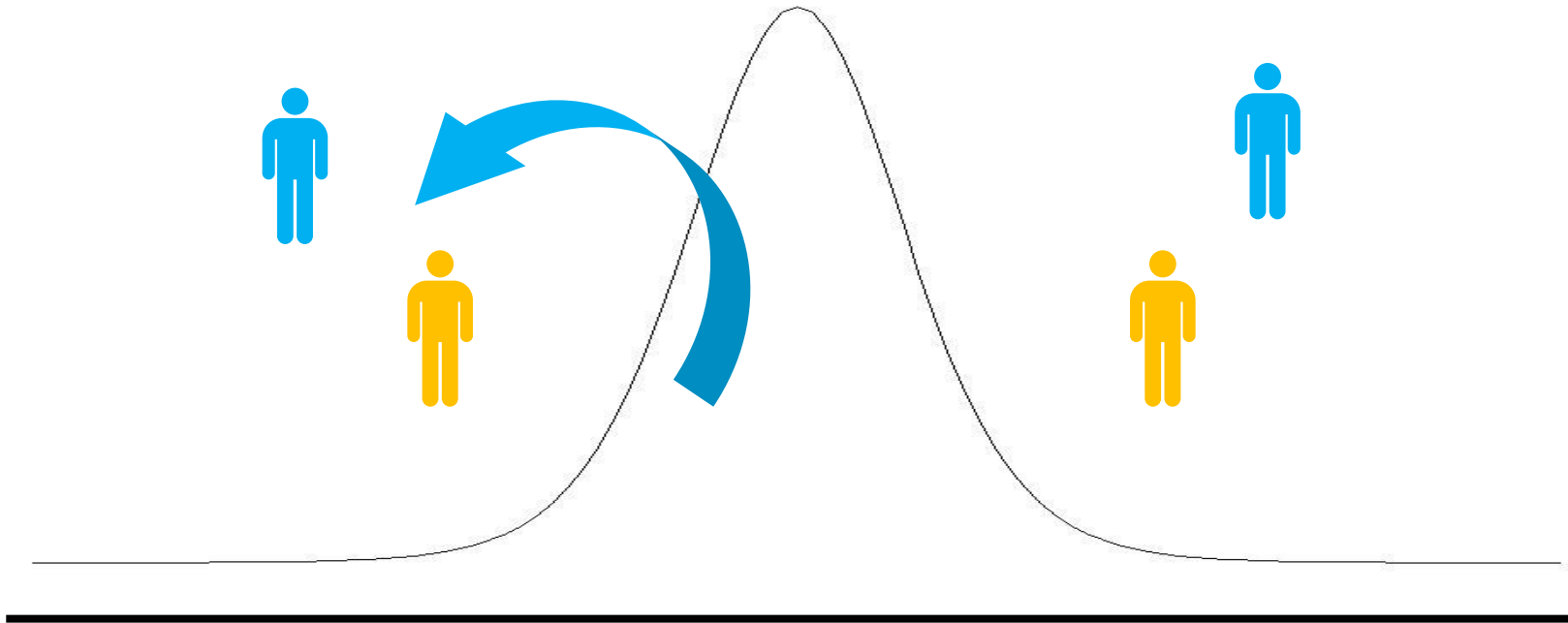
Means Delta Degrees of Freedom. The divisor used in calculations is `N - ddof`, where `N` represents the number of elements. By default `ddof` is zero.

AI/BigData 인텐시브 과정

Section 1. 기초 통계

Section 1-4. 모집단과 표본

모집단(population)과 표본(sample)



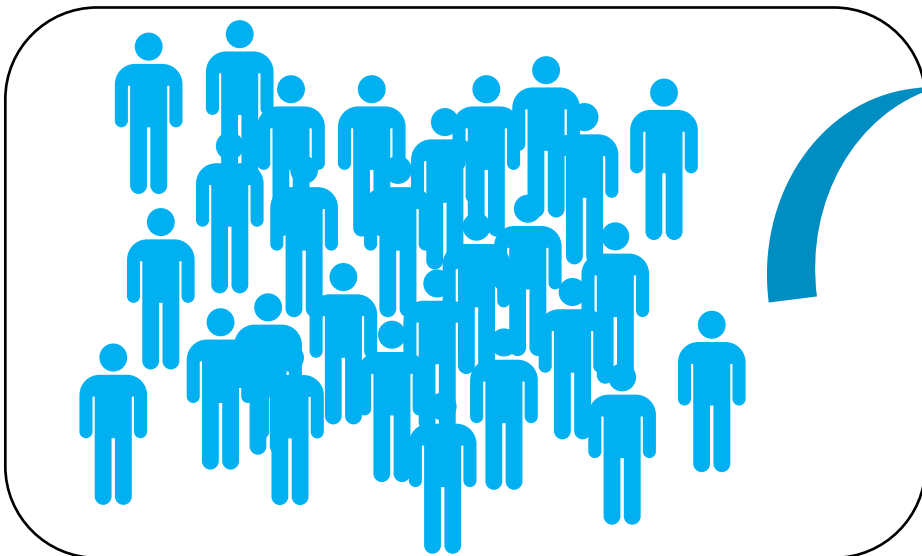
모집단(population) : 관심대상 전체 집합

표본(sample) : 모집단의 부분 집합

모집단(population)과 표본(sample)

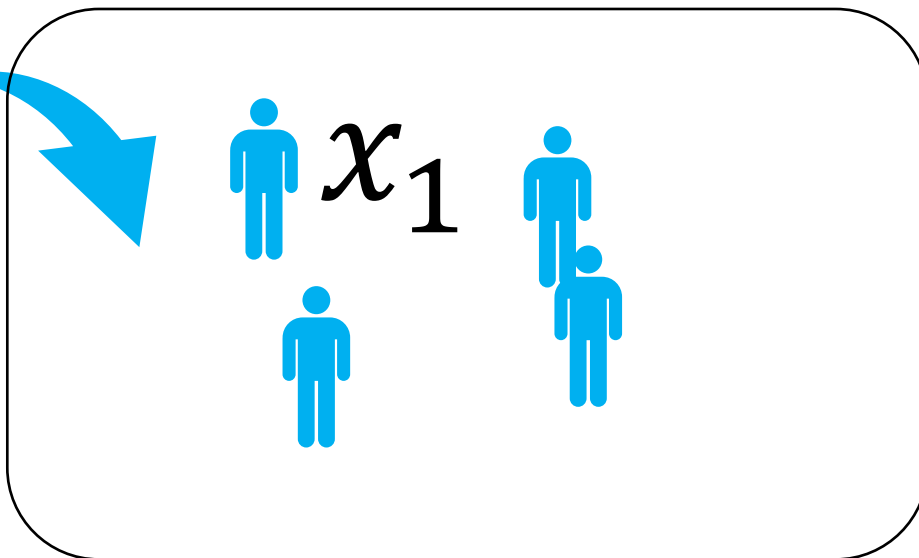
* 표본이지만 모집단을 착각하는 경우 많음
대한민국 주민데이터? → 표본.
주관적측량한 자연인
있을 수 있음

모집단(population)



경기도에 살고 있는
삼성전자 고객의 나이 모음

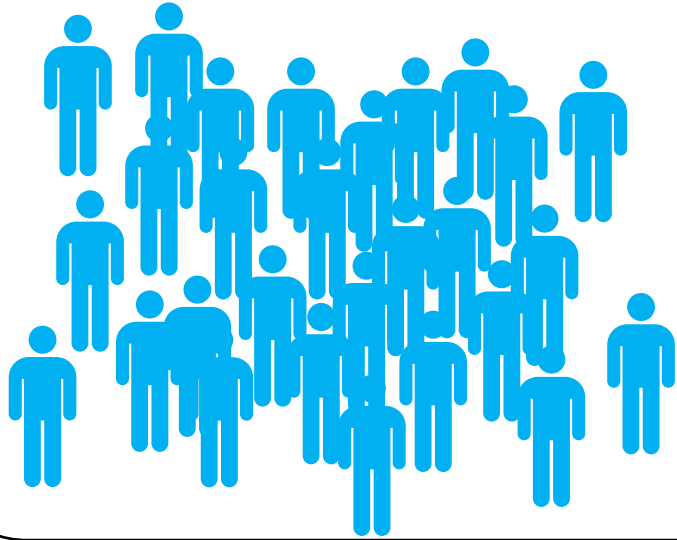
표본(sample)



모집단에서 추출된
삼성전자 고객의 나이 모음

모집단(population)과 표본(sample)

모집단(population)



경기도에 살고 있는
삼성전자 고객의 나이 모음

모평균(population mean) = 모집단의 평균

μ 뮤 $\rightarrow$ 평균

모분산(population variance) = 모집단의 분산

σ^2 시그마 제곱

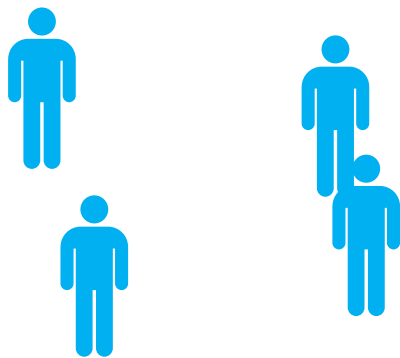
모표준편차(population standard deviation) = 모집단의 표준편차

σ 시그마

모집단(population)과 표본(sample)

원시에서 활용하는 데이터는
대부분이 정형라서, (표본 데이터)
 $\bar{x}, s^2, s$ 로 표기해야함

표본(sample)



모집단에서 추출된
삼성전자 고객의 나이 모음

표본 평균(sample mean) = 표본의 평균

$\bar{x}$ → 관찰 가능

표본 분산(sample variance) = 표본의 분산

s^2

표본 표준편차(sample standard deviation) = 표본의 표준편차

s

(if 모집단을 지정한다면 모든 값을 알아야 한다)

모수(population parameter)

≠ 데이터개수

모수(population parameter)

 μ

= 모집단의 특성을 나타내는 대표값

 σ^2 σ

ex) 모평균, 모분산, 모표준편차

위치특성

흩어짐특성

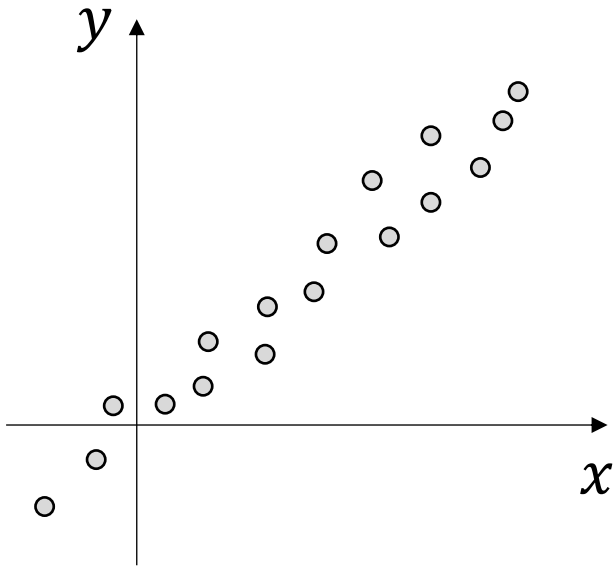
AI/BigData 인텐시브 과정

Section 1. 기초 통계

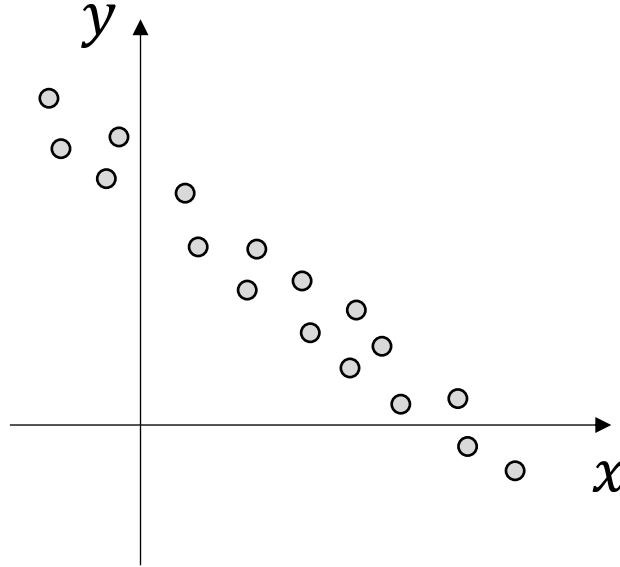
Section 1-5. 공분산과 상관계수

상관 관계

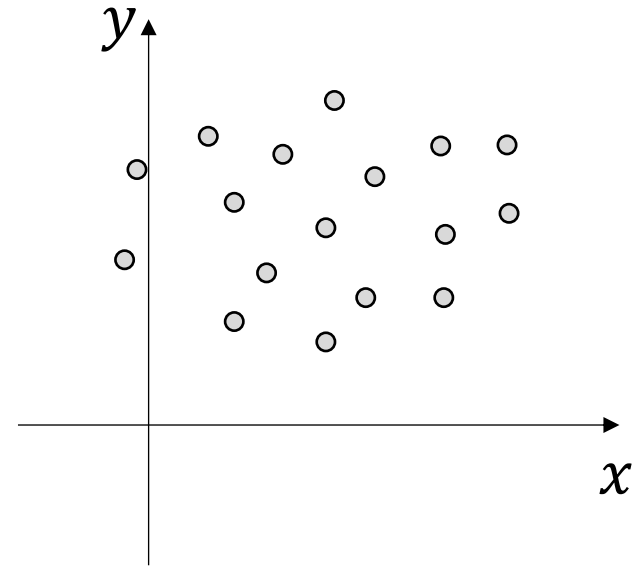
변수들 간의 상관관계를 보기위함



양의 상관관계
positive correlation



음의 상관관계
negative correlation



상관관계 없음
no correlation

공분산(covariance)

$$Cov(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

(Handwritten notes: X is circled with "확률변수" (random variable); μ_X is circled with "모평균 (다항식 포함)" (population mean (including polynomial)).

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

(Handwritten notes: $n-1$ is circled with "표본공분산" (sample covariance) and "n으로 잘 안나올 n-1로 나눌" (won't come out well with n, divide by n-1); $\bar{x}$ and $\bar{y}$ are circled.

시험법) 시험에 원으로 계산하는 거 나열지도

공분산(covariance)

$$Cov(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

x	y
1	3
2	5
6	4

$$\bar{x} = 3 \quad \bar{y} = 4$$

$$Cov(X, Y) = \frac{1}{3-1} [\overset{\text{내제}}{(1-3)(3-4)} + (2-3)(5-4) + (6-3)(4-4)]$$

$$= \frac{1}{3-1} [(-2)(-1) + (-1)(1) + (3)(0)]$$

$$= \frac{1}{2} [2 - 1 + 0] = \frac{1}{2}$$

2x2에 들어가는 숫자 중 하나를 구한 것

공분산(covariance) 벡터를 구하는데 계산빠름.

$$Cov(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

x	y
1	3
2	5
6	4

$$\bar{x} = 3 \quad \bar{y} = 4$$

x	$\bar{x}$	$x - \bar{x}$
1	3	-2
2	3	-1
6	3	3

y	$\bar{y}$	$y - \bar{y}$
3	4	-1
5	4	1
4	4	0

공분산(covariance)

$$\begin{matrix} x & y \\ \begin{bmatrix} 1 \\ 2 \\ 6 \end{bmatrix} & \begin{bmatrix} 3 \\ 5 \\ 4 \end{bmatrix} \end{matrix} = X, \quad \begin{bmatrix} 3 & 4 \\ 3 & 4 \\ 3 & 4 \end{bmatrix} = \bar{X}$$

$$Cov(X) = \frac{1}{n-1} (X - \bar{X})^T (X - \bar{X})$$

내적.

$$= \frac{1}{n-1} X^T X$$

core function

scaler

$X=0$ 이면
: 많은 경우에 데이터가 0을
통계 평균을 0으로 만든다

⇒ 2x2 행렬이 됨
대칭행렬임

공분산(covariance)

대각선 (\) 에 있는 값은 분산값이 될
대칭행렬임

$x - \bar{x}$
-2
-1
3

$y - \bar{y}$
-1
1
0

$$\frac{1}{2} \left[\begin{array}{|c|c|c|} \hline -2 & -1 & 3 \\ \hline -1 & 1 & 0 \\ \hline \end{array} \quad \begin{array}{|c|c|} \hline -2 & -1 \\ \hline -1 & 1 \\ \hline 3 & 0 \\ \hline \end{array} \right]$$

$(X - \bar{X})^T$
 $(X - \bar{X})$

공분산(covariance)

In [36]:

1 df

Out[36]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222
...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273

506 rows × 14 columns

In [35]:

1 X = df[['CRIM', 'NOX', 'RM', 'AGE', 'TAX']]

In [34]:

1 X

Out[34]:

	CRIM	NOX	RM	AGE	TAX
0	0.00632	0.538	6.575	65.2	296
1	0.02731	0.469	6.421	78.9	242
2	0.02729	0.469	7.185	61.1	242
3	0.03237	0.458	6.998	45.8	222
4	0.06905	0.458	7.147	54.2	222
...	...	...	...	...	...
501	0.06263	0.573	6.593	69.1	273
502	0.04527	0.573	6.120	76.7	273
503	0.06076	0.573	6.976	91.0	273
504	0.10959	0.573	6.794	89.3	273
505	0.04741	0.573	6.030	80.8	273

506 rows × 5 columns

공분산(covariance)

행은 변인로 볼것이야

```
1 cov_x = np.cov(X, rowvar=0)
2 print(cov_x)
```

```
[[ 7.39865782e+01  4.19593894e-01 -1.32503785e+00  8.54053223e+01
   8.44821538e+02]
 [ 4.19593894e-01  1.34276357e-02 -2.46034495e-02  2.38592720e+00
   1.30462855e+01]
 [-1.32503785e+00 -2.46034495e-02  4.93670850e-01 -4.75192919e+00
  -3.45834478e+01]
 [ 8.54053223e+01  2.38592720e+00 -4.75192919e+00  7.92358399e+02
   2.40269012e+03]
 [ 8.44821538e+02  1.30462855e+01 -3.45834478e+01  2.40269012e+03
  2.84047595e+04]]
```

In [81]: ▶

```
1 cov_x = X.cov()
2 cov_x
```

Out[81]:

	CRIM	NOX	RM	AGE	TAX
CRIM	73.986578	0.419594	-1.325038	85.405322	844.821538
NOX	0.419594	0.013428	-0.024603	2.385927	13.046286
RM	-1.325038	-0.024603	0.493671	-4.751929	-34.583448
AGE	85.405322	2.385927	-4.751929	792.358399	2402.690122
TAX	844.821538	13.046286	-34.583448	2402.690122	28404.759488

NUMPY에서는 행을 feature로 보는데
X.T로 바꾸어야함

```
1 cov_x = np.cov(X.T)
2 print(cov_x)
```

```
[[ 7.39865782e+01  4.19593894e-01 -1.32503785e+00  8.54053223e+01
   8.44821538e+02]
 [ 4.19593894e-01  1.34276357e-02 -2.46034495e-02  2.38592720e+00
   1.30462855e+01]
 [-1.32503785e+00 -2.46034495e-02  4.93670850e-01 -4.75192919e+00
  -3.45834478e+01]
 [ 8.54053223e+01  2.38592720e+00 -4.75192919e+00  7.92358399e+02
   2.40269012e+03]
 [ 8.44821538e+02  1.30462855e+01 -3.45834478e+01  2.40269012e+03
  2.84047595e+04]]
```

공분산(covariance)

$$-\infty < Cov(X, Y) < \infty$$

In [81]: ▶

```
1 cov_x = X.cov()  
2 cov_x
```

Out[81]:

	CRIM	NOX	RM	AGE	TAX
CRIM	73.986578	0.419594	-1.325038	85.405322	844.821538
NOX	0.419594	0.013428	-0.024603	2.385927	13.046286
RM	-1.325038	-0.024603	0.493671	-4.751929	-34.583448
AGE	85.405322	2.385927	-4.751929	792.358399	2402.690122
TAX	844.821538	13.046286	-34.583448	2402.690122	28404.759488

공분산은 데이터 값의 단위에 영향을 받는다

숫자가 커지면 공분산도 같이 커진다

cm, m 등...

피어슨 상관계수(Pearson Correlation Coefficient)

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}$$

Cov(X, Y) ~ core function

$\sigma_X \sigma_Y$ ~ scaler

σ_X ↓
X의 표준편차

코사인 유사도도 scaler 적용해서 값의 범위를 제한하였음

피어슨 상관계수(Pearson Correlation Coefficient)

$$-1 \leq \text{Corr}(X, Y) \leq 1$$

$-1 \leq \text{Corr}(X, Y) < -0.6$	강한 음의 상관 관계	0.6이라는 절대적인 기준은 없음. 참고용.
$-0.6 < \text{Corr}(X, Y) < 0$	약한 음의 상관 관계	
$0 < \text{Corr}(X, Y) < 0.6$	약한 양의 상관 관계	
$0.6 < \text{Corr}(X, Y) \leq 1$	강한 양의 상관 관계	

피어슨 상관계수(Pearson Correlation Coefficient)

X, Y 가 서로 상관관계가 없으면 $\Rightarrow \text{Corr}(X, Y) = 0$

$\text{Corr}(X, Y) = 0 \Rightarrow X, Y$ 가 서로 상관관계가 없다

역이 성립하지 않는다. 왜???

상관계수가 0인데 관계가 있는 경우



: 상관계수는 0이지만 관계성이 있을 수.

피어슨 상관계수(Pearson Correlation Coefficient)

In [82]: ▶

```
1 corr_x = np.corrcoef(X.T)
2 print(corr_x)
```

```
[[ 1.          0.42097171 -0.2192467  0.35273425  0.58276431]
 [ 0.42097171  1.          -0.30218819  0.7314701  0.6680232 ]
 [-0.2192467  -0.30218819  1.          -0.24026493 -0.29204783]
 [ 0.35273425  0.7314701  -0.24026493  1.          0.50645559]
 [ 0.58276431  0.6680232  -0.29204783  0.50645559  1.          ]]
```

In [84]: ▶

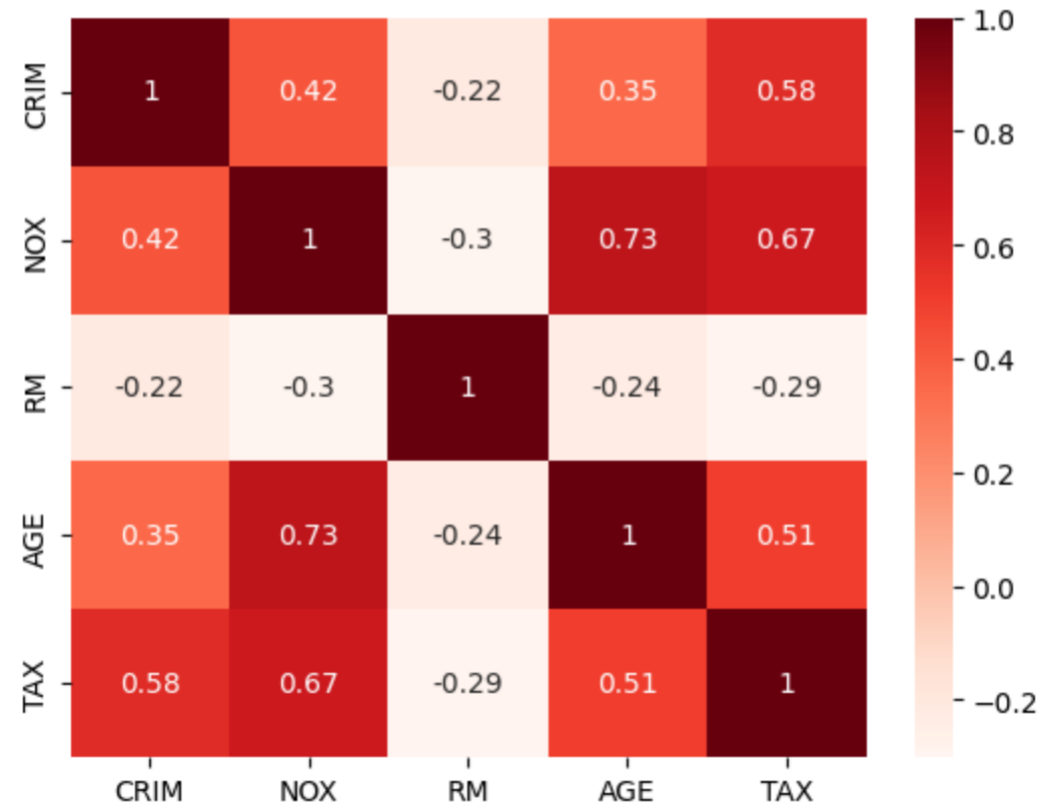
```
1 corr_x = X.corr()
2 corr_x
```

Out[84]:

	CRIM	NOX	RM	AGE	TAX
CRIM	1.000000	0.420972	-0.219247	0.352734	0.582764
NOX	0.420972	1.000000	-0.302188	0.731470	0.668023
RM	-0.219247	-0.302188	1.000000	-0.240265	-0.292048
AGE	0.352734	0.731470	-0.240265	1.000000	0.506456
TAX	0.582764	0.668023	-0.292048	0.506456	1.000000

```
1 import seaborn as sns
2
3 sns.heatmap(corr_x, annot=True, cmap='Reds')
```

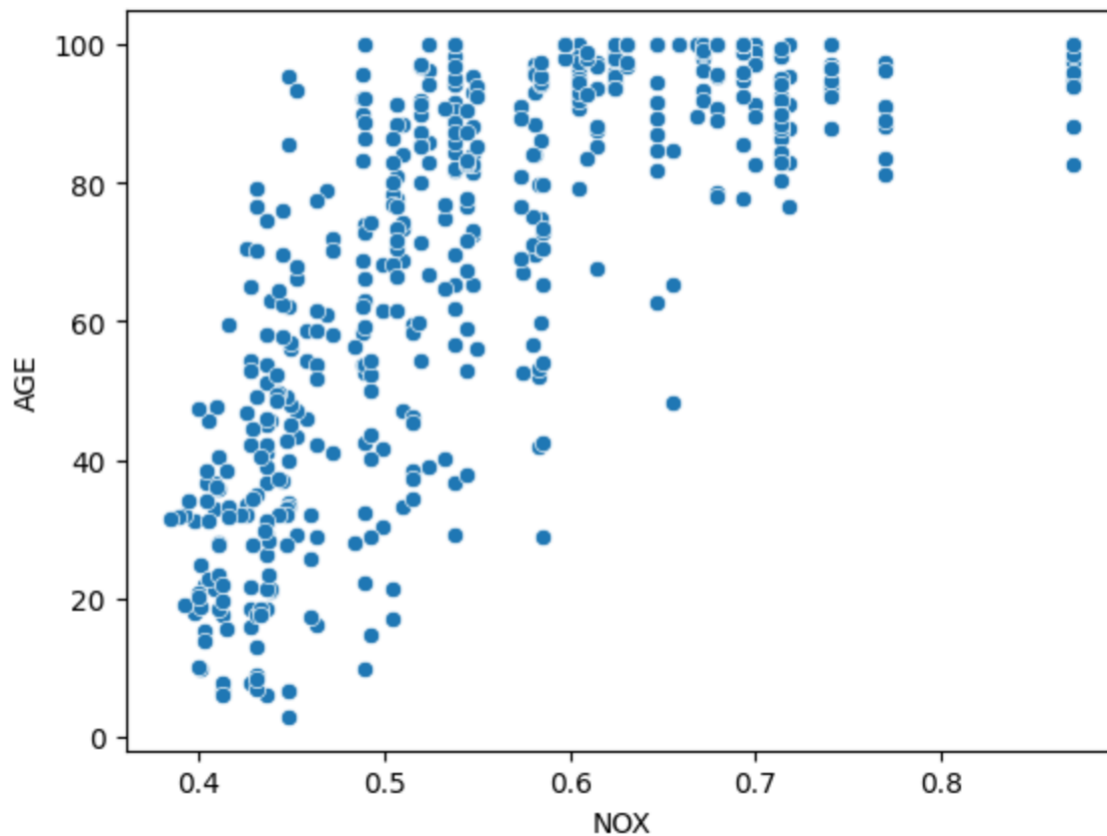
<Axes: >



피어슨 상관계수(Pearson Correlation Coefficient)

```
In [89]: 1 sns.scatterplot(data=df, x="NOX", y='AGE')
```

```
Out[89]: <Axes: xlabel='NOX', ylabel='AGE'>
```



AGE를 증가시키기 위해
NOX를 증가시킨다? → 이상함.

피어슨 상관계수(Pearson Correlation Coefficient)

상관 관계가 존재하면 ^{원인과 결과}인과 관계가 존재한다?



회사에서는 인과관계 알고 싶어하지만
현관하는 방법중에 인과관계를 볼수 있는건 X.

상관계수 높다면... 원인일수도 있다고
조심스럽게 접근해야함

감사합니다. * 회귀분석의 계수값과 상관계수는 서로 관계없다.

* 상관계수와 R^2 는 다름

(ML하기 전 봐둬야)

- 행렬상대.
- 교차상대.
- 상관계수.

(feature가 부족할때 낮아지는 이론은
별로 없음..)

Q & A

행수가 많아도 R^2 와 correlation은 다름. 이제 귀하는 두가지게가 다름

